



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Balanced Parentheses Using Stack

A simple technical presentation on compiler syntax analysis

Data Structure: Stack (LIFO)

Application: Checking (), {}, and []

CSA0315-DATA STRUCTURES

Name: M Mohanasiva vikasvanth reddy
Reg no:192573011

Introduction

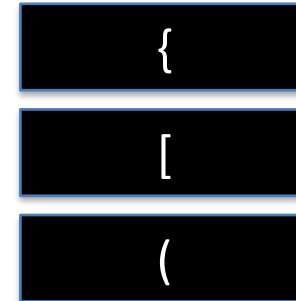
- A compiler checks whether a program follows correct syntax.
- Brackets are an important part of program syntax.
- `()`, `{}`, and `[]` must be properly matched.
- A Stack provides a simple and efficient way to check them.
- Balanced brackets are especially important in nested code blocks.
- This check is a basic part of syntax validation.

Problem Statement

- Identify whether brackets in a program are balanced.
- Detect missing, extra, or incorrect brackets.
- Ensure every opening bracket has a matching closing bracket.
- Choose a data structure that supports this operation efficiently.
- The checker should process brackets in their correct order.
- Invalid input should be reported immediately.

Why Use a Stack?

- Stack follows LIFO — Last In, First Out.
- The most recently opened bracket must be closed first.
- Opening brackets are stored using PUSH.
- Matching brackets are removed using POP.
- Peek is used to inspect the top bracket without removing it.
- A stack can handle deeply nested expressions naturally.



Working Principle

1. Read the program character by character.
2. Opening bracket → PUSH into the stack.
3. Closing bracket → CHECK the top element.
4. If it matches → POP the element.
5. If it does not match → report a syntax error.

Ignore normal characters such as letters, numbers, and operators.

Continue scanning until the complete expression is checked.

Example:

Input: `if (x > 5) { printf("Hello"); }`

`( → PUSH      ) → POP`

`{ → PUSH      } → POP`

Final Stack: EMPTY

Result: BALANCED PROGRAM

Example of nesting: `{ [ ( ) ] }`.

A mismatch at any stage makes the expression invalid.

Algorithm & Complexity

- Start with an empty stack.
- Scan each character in the input.
- Push every opening bracket.
- Check and pop for every closing bracket.
- Mismatch or empty-stack condition → Syntax Error.
- Empty stack at the end → Balanced.
- Time Complexity: $O(n)$ | Space Complexity: $O(n)$
- The algorithm scans the input only once.
- Performance remains efficient even for long expressions.

Conclusion

- Stack is the most suitable structure for balanced-parentheses checking.
- Its LIFO behavior matches the order of nested brackets.
- It can detect missing, extra, and mismatched brackets.
- The method is simple, fast, and easy to implement.
- Therefore, Stack plays an important role in compiler syntax analysis.
- The approach is useful in compilers, editors, and code validators.
- It demonstrates how a simple data structure solves a practical programming problem.

THANK YOU